LAB REPORT VIVA 2

Team 5: Business Buzz

No.	Matric No.	Name
1.	24067901	Shaif Ahmad Bin Tufail Ahmad
2.	23121478	Tanvir Hossain
3.	24004539	Vosshindran A/L Thiyagaraja
4.	24064422	Sifat Ul Islam Babor
5.	23002044	Karl Xavier Franky
6.	23103329	Osman Mohammedelamin Elsharief Ahmed

Question 1

Section	Description
1. Problem	Write a method called findLongestPalindromicSubstring to find the longest palindromic substring in a given string. A palindromic substring is a substring of the given string that reads the same forwards and backwards. The method should return the longest palindromic substring found within the input string. In the main method, get input from the user and then display the longest palindromic substring within the input string using the findLongestPalindromicSubstring
2. Solution	To find the longest palindromic substring within a given string, we can use the following approach: 1. Iterate through each character in the input string. 2. For each character, check for both odd-length and even-length palindromes that can be formed using that character as the center. 3. To check for an odd-length palindrome, expand outwards from the current character, comparing characters on both sides until they no longer match. 4. To check for an even-length palindrome, expand outwards from the current character and the next character, comparing characters on both sides until they no longer match. 5. Keep track of the longest palindromic substring found so far, and update it if a longer one is discovered. 6. After checking all characters, return the longest palindromic substring.
3. Sample Input & Output	**Sample Input Output:** ***Example 1*** *Enter string: bananas* *Longest Palindromic Substring: anana* ***Example 2*** *Enter string: racecar* *Longest Palindromic Substring: racecar* ***Example 3*** *Enter string: abaabba* *Longest Palindromic Substring: abba*

4. Source Code	Link: SIFAT UL ISLAM BABOR
	import java.util.Scanner;
	
	public class Main {
	  public static String findLongestPalindromicSubstring(String str) {
	  if (str == null || str.length() < 1) {
	  return "";
	  }

```
    String longestPalindrome = "";

    for (int i = 0; i < str.length(); i++) {
        for (int j = i; j < str.length(); j++) {

            String currentSubstring = str.substring(i, j + 1);


            if (isPalindrome(currentSubstring)) {

                if (currentSubstring.length() > longestPalindrome.length()) {
                    longestPalindrome = currentSubstring;
                }
            }
        }
    }

    return longestPalindrome;
}


private static boolean isPalindrome(String str) {
    int left = 0;
    int right = str.length() - 1;

    while (left < right) {
        if (str.charAt(left) != str.charAt(right)) {
            return false;
        }
        left++;
        right--;
    }

    return true;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);


    while (true) {
        System.out.print("Enter string (or type 'exit' to quit): ");
        String inputString = scanner.nextLine().trim();


        if (inputString.equalsIgnoreCase("exit")) {
            break;
```

```
            }


            if (!inputString.matches("[a-z]+")) {
                System.out.println("Invalid input. Please enter a string with only lowercase
letters.");
                continue;
            }

            String longestPalindrome = findLongestPalindromicSubstring(inputString);
            System.out.println("Longest Palindromic Substring: " + longestPalindrome);
        }

        System.out.println("Program ended.");
        scanner.close();
    }
}
```

Question 2

Section	Description
1. Problem	Create a Java program that accepts two lines of input representing integer arrays, merges them, and outputs a sorted array without duplicates. The program must include the following: • A method `parseArray` to convert input strings into integer arrays. • A method `mergeArray` to merge the two arrays, sort them in ascending order, and remove duplicates. • The use of `ArrayList` or other dynamic collections is prohibited.

2. Solution	To solve the problem:
	
	1. **Input Parsing:**
	  a. Use the `parseArray` method to convert comma-separated strings into integer arrays. This involves splitting the string and parsing each segment as an integer.
	2. **Array Merging:**
	  a. Use the `mergeArray` method to merge the two input arrays into one.
	  b. Sort the merged array using `Arrays.sort`.
	  c. Remove duplicate values by iterating through the sorted array and keeping only unique numbers.
	
	`parseArray` Method
	
	This method is responsible for converting a user-provided string into an integer array.
	
	1. **Input**: A string that contains integers separated by commas, such as `"1, 2, 3"`.
	2. **Split**: Use the `split` method with a comma, as the delimiter to separate the numbers into a string array.
	3. **Trim and Convert**: Iterate through the string array, trim any extra spaces, and convert each element into an integer.
	4. **Store in Array**: Add each integer to the corresponding index in a new integer array.
	5. **Return**: The method returns the final integer array.
	
	Example:
	
	Input: `"1, 2, 3"`
	
	Output: `[1, 2, 3]`
	
	`mergeArray` Method
	
	This method combines two integer arrays into one, removes duplicate elements, and sorts the resulting array in ascending order.

1. **Input**: Two integer arrays.
2. **Merge Arrays**: Create a new array large enough to hold all elements of both arrays and copy the elements.
3. **Remove Duplicates**: Use a `HashSet` to remove duplicates since sets automatically discard repeated values.
4. **Sort the Array**: Convert the `HashSet` back into an array, then sort it using `Arrays.sort`.
5. **Return**: Return the sorted array without duplicates.

Example:

Input: `array1 = [1, 2, 3]`
`array2 = [6, 3, 4]`

Output: `[1, 2, 3, 4, 6]`

3. Sample Input & Output

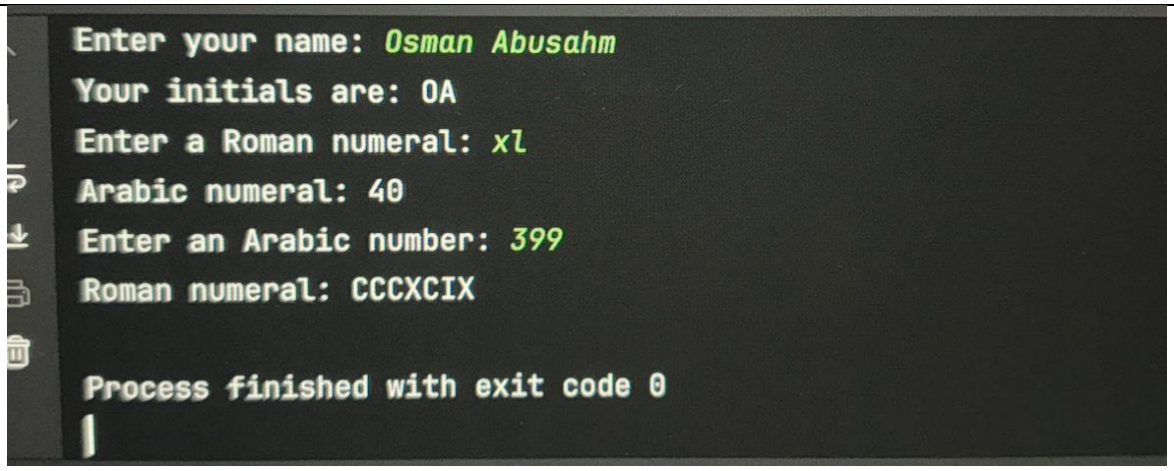
```
Array 1: 1,2,3
Array 2: 3,4,5
[1, 2, 3, 4, 5]
```

4. Source Code

```
import java.util.*;

public class V02Q02 {
   public static void main(String[] args){
      Scanner scan = new Scanner(System.in);

      System.out.print("Array 1: ");
      String input1 = scan.nextLine(); // Read the first input as a string
      int[] array1 = parseArray(input1); // Parse the input string into an integer array

      System.out.print("Array 2: ");
      String input2 = scan.nextLine(); // Read the second input as a string
      int[] array2 = parseArray(input2);// Parse the input string into an integer array

      // Merge the two arrays
      int[] mergedArray = mergeArray(array1, array2);

      // Print the final merged array
      System.out.println(Arrays.toString(mergedArray));
   }

   /*  Parses a comma-separated string into an integer array. */

   public static int[] parseArray(String input){

      String[] parts = input.split(","); // Split the input string by commas
```

```
        int[] result = new int[parts.length];
        // Create an integer array to store the parsed values

        // Convert each part into an integer
        for (int i = 0; i < parts.length; i++){
            result[i] = Integer.parseInt(parts[i].trim());
            // Trim whitespace and parse as integer
        }
        return result; // Return the parsed array
    }

    /* Merges two integer arrays, sorts them in ascending order, and removes duplicate
values. */

    public static int[] mergeArray(int[] array1, int[] array2) {
        // Create a temporary array to hold the combined elements of both arrays
        int[] tempArray = new int[array1.length + array2.length];

        // Copy the elements of the first array into the temporary array
        System.arraycopy(array1, 0, tempArray, 0, array1.length);

        // Copy the elements of the second array into the temporary array
        System.arraycopy(array2, 0, tempArray, array1.length, array2.length);

        // Sort the combined array in ascending order
        Arrays.sort(tempArray);

        // Create an array to store unique elements (no duplicates)
        int[] uniqueArray = new int[tempArray.length];
        int index = 0; // Index to keep track of unique values

        // Iterate through the sorted array and add unique elements to uniqueArray
        for (int i = 0; i < tempArray.length; i++) {
         // Only add the element if it is the first element or different from the previous
            if (i == 0 || tempArray[i] != tempArray[i - 1]) {
                uniqueArray[index++] = tempArray[i];
            }
        }
        // Return a copy of the unique array with the exact number of unique elements
        return Arrays.copyOf(uniqueArray, index);
    }
}
```

Question 3

Section	Description
1. Problem	Write a program that can translate between Arabic numerals and Roman numerals to make it more accessible for everyone
2. Solution	1. **Generate Initials** (generateInitials): • Splits a full name by spaces and retrieves the first letter of each name. 2. **Convert Roman Numeral to Arabic** (convertArabics): • Iterates through the Roman numeral string from right to left, applying subtractive rules where needed. 3. **Convert Arabic Numeral to Roman** (convertRomans): • Iteratively subtracts the largest possible Roman numeral value while appending its symbol to the result. 4. **Error Handling**: • Invalid inputs (e.g., wrong Roman numerals or out-of-range Arabic numbers) throw appropriate exceptions with error messages. 5. **Demonstration in** main: • Collects user input, processes it, and displays results.
3. Sample Input & Output	
4. Source Code	

```java
import java.util.Scanner;
import java.util.TreeMap;

public class Main {

    // TreeMap to map Roman numerals to Arabic values
    private static final TreeMap<Character, Integer> romanToArabicMap = new TreeMap<>();
    private static final TreeMap<Integer, String> arabicToRomanMap = new TreeMap<>();

    static {
        // Roman to Arabic mappings
        romanToArabicMap.put('I', 1);
        romanToArabicMap.put('V', 5);
        romanToArabicMap.put('X', 10);
        romanToArabicMap.put('L', 50);
```

```java
        romanToArabicMap.put('C', 100);
        romanToArabicMap.put('D', 500);
        romanToArabicMap.put('M', 1000);

        // Arabic to Roman mappings
        arabicToRomanMap.put(1000, "M");
        arabicToRomanMap.put(900, "CM");
        arabicToRomanMap.put(500, "D");
        arabicToRomanMap.put(400, "CD");
        arabicToRomanMap.put(100, "C");
        arabicToRomanMap.put(90, "XC");
        arabicToRomanMap.put(50, "L");
        arabicToRomanMap.put(40, "XL");
        arabicToRomanMap.put(10, "X");
        arabicToRomanMap.put(9, "IX");
        arabicToRomanMap.put(5, "V");
        arabicToRomanMap.put(4, "IV");
        arabicToRomanMap.put(1, "I");
    }

    // Method to generate initials from user input
    public static String generateInitials(String name) {
        String[] parts = name.split(" ");
        StringBuilder initials = new StringBuilder();
        for (String part : parts) {
            if (!part.isEmpty()) {
                initials.append(part.charAt(0));
            }
        }
        return initials.toString().toUpperCase();
    }

    // Method to convert Roman numerals to Arabic numbers
    public static int convertArabics(String roman) {
        int result = 0;
        for (int i = 0; i < roman.length(); i++) {
            int current = romanToArabicMap.getOrDefault(roman.charAt(i), 0);
            if (i < roman.length() - 1 && current < romanToArabicMap.getOrDefault(roman.charAt(i + 1), 0)) {
                result -= current; // Subtraction case
            } else {
                result += current; // Normal addition case
            }
        }
        return result;
    }

    // Method to convert Arabic numbers to Roman numerals
    public static String convertRomans(int number) {
        StringBuilder roman = new StringBuilder();
        for (Integer key : arabicToRomanMap.descendingKeySet()) {
            while (number >= key) {
                roman.append(arabicToRomanMap.get(key));
                number -= key;
            }
        }
        return roman.toString();
    }

    // Main method for testing
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```java
        // Get user's name and generate initials
        System.out.print("Enter your name: ");
        String name = scanner.nextLine();
        String initials = generateInitials(name);
        System.out.println("Your initials are: " + initials);

        // Convert Roman numeral to Arabic number
        System.out.print("Enter a Roman numeral: ");
        String roman = scanner.nextLine().toUpperCase();
        try {
            int arabic = convertArabics(roman);
            System.out.println("Arabic numeral: " + arabic);
        } catch (Exception e) {
            System.out.println("Invalid Roman numeral input.");
        }

        // Convert Arabic number to Roman numeral
        System.out.print("Enter an Arabic number: ");
        int arabic;
        try {
            arabic = scanner.nextInt();
            if (arabic <= 0 || arabic > 3999) {
                System.out.println("Invalid input. Arabic numbers should be between 1 and 3999.");
            } else {
                String romanResult = convertRomans(arabic);
                System.out.println("Roman numeral: " + romanResult);
            }
        } catch (Exception e) {
            System.out.println("Invalid Arabic number input.");
        }

        scanner.close();
    }
}
```

Question 4

Section	Description
1. Problem	Create a function format(number, width) that adds zeros to the left of a number until it matches the width. If the number is longer than the width, return it as is. Write a program that asks the user for a number and width, uses the function, and shows the result.
2. Solution	**Function (format)**: • The function takes two inputs: number and width. • It uses String.format() to format the number by padding with zeros if the length is less than the width. **Check the Length**: • The String.Format("%0" + width + "d", number) handles the length check by formatting the number to the desired width with leading zeros if necessary. **User Input and Display**: • The program prompts the user to enter a number and a width. • It then calls the format() function and displays the result.
3. Sample Input & Output	--- exec:3.1.0:exec (default-cli) @ VIVA2Q4 --- Enter an integer: 34 Enter the width: 5 Formatted number: 00034 ---------- BUILD SUCCESS ---------- Total time: 5.113 s Finished at: 2024-12-08T20:42:29+08:00 ----------

4. Source Code	Link: shaifq4
	public static String format(int number, int width) {
	//%: This starts the format specifier,
	// 0: : This is the padding character
	//d: Represents a whole number as a decimal integer.
	return String.format("%0" + width + "d", number);
	}
	
	public static void main(String[] args) {
	Scanner scanner = new Scanner(System.in);
	
	System.out.print("Enter an integer: ");
	int number = scanner.nextInt();

```
        System.out.print("Enter the width: ");
        int width = scanner.nextInt();

            String formattedNumber = format(number, width);
        System.out.println("Formatted number: " + formattedNumber);

        scanner.close();
    }
}
```

Question 5

Section	Description
1. Problem	The main method of the program is given. The task given is to complete the program using methods that were given so that it functions as intended.
2. Solution	1. **`printFruit(String[] fruits)`**: • Loop through the `fruits` array. • Print each fruit in the list with a corresponding number (0, 1, 3, etc.). 2. **`getIdxFruit(String[] fruits, String selectedFruit)`**: • Loop through the `fruits` array and compare each fruit to the selected one. • Return the index of the selected fruit if found; otherwise, return -1. 3. **`haveProduct(String[] fruits, int fruitIdx)`**: • Check if the given `fruitIdx` is a valid index within the `fruits` array. • Return `true` if the index is valid, otherwise return `false`. 4. **`haveStock(int fruitIdx, int[] storage, int quantity)`**: • Check if the stock at the given `fruitIdx` is greater than or equal to the requested `quantity`. • Return `true` if enough stock is available, otherwise return `false`. 5. **`generateBill(String[] fruits, int fruitIdx, double[] price, int quantity)`**: • Calculate the total price by multiplying the quantity and price per unit. • Print a formatted receipt, displaying the product name, quantity, price per unit, and the total price.
3. Sample Input & Output	1. Apple 2. Grape 3. Banana 4. Mango 5. Orange 6. Strawberry 7. Blueberry Please enter the fruit you want to buy: orange Unit or Box: 23 Kindly Checkout here: ---------- ----------Fresh Mart Receipt---------- ---------- Product Quantity Price per Unit(RM) Orange 23 3.0 ----------

```
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: Orange
Unit or Box: 90
Sorry, we do not have enough stock for Orange. Please come again later!
```

```
1. Apple
2. Grape
3. Banana
4. Mango
5. Orange
6. Strawberry
7. Blueberry
Please enter the fruit you want to buy: watermelon
Unit or Box: 10
Sorry we currently do not offer this product.
```

4. Source Code

Link: freshMart.java[Karl]

```
package freshmart;

import java.util.Scanner;

public class freshMart {


    public static void printFruit(String[] fruits){
        String[] list=fruits;
        int num=1;
        for(String n : list){
            System.out.println(num + ". " + n);
            num++;
        }
    }

    public static int getIdxFruit(String[] fruits, String selectedFruit){
        String select = selectedFruit.replaceAll(" ","");
        int i;
        for(i=0;i<7;i++){
            if(fruits[i].equalsIgnoreCase(select)){
                return i;
            }
        }
        return -1;
    }

    public static boolean haveProduct(String[] fruits, int fruitIdx){
            return fruitIdx>=0 && fruitIdx<fruits.length;
    }

    public static boolean haveStock(int fruitIdx, int[] storage, int quantity){
            return storage[fruitIdx]>=quantity;
    }
```

```
    public static void generateBill(String[] fruits, int fruitIdx, double[]price, int
quantity){
        double tot=quantity*price[fruitIdx];
        System.out.println("------------------------------------------------------------");
        System.out.println("-----------------Fresh Mart Receipt-------------------");
        System.out.println("------------------------------------------------------------");
        System.out.println("Product                 Quantity  Price per Unit(RM)");
        System.out.println(fruits[fruitIdx] + "                    " + quantity + "               " +
price[fruitIdx]);
        System.out.println("------------------------------------------------------------");
        System.out.printf("Total Price: RM%.2f\n", tot);
        System.out.println("------------------------------------------------------------");
    }

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);

        String[] fruits = {"Apple", "Grape", "Banana", "Mango", "Orange",
"Strawberry", "Blueberry"};
        int[] storage = {45, 57, 75, 34, 75, 65, 78};
        double[] price = {5.00, 10.00, 2.50, 6.00, 3.00, 15.00, 12.00};

        printFruit(fruits);

        System.out.print("Please enter the fruit you want to buy: ");
        String fruitSelect = in.nextLine();

        System.out.print("Unit or Box: ");
        int quantity = in.nextInt();

        int fruitIdx = getIdxFruit(fruits, fruitSelect);

        if (!haveProduct(fruits, fruitIdx)) {
            System.out.println("Sorry we currently do not offer this product.");
        } else if (!haveStock(fruitIdx, storage, quantity)) {
            System.out.println("Sorry, we do not have enough stock for " + fruits[fruitIdx]
+ ". Please come again later!");
        } else {
            System.out.println("\nKindly Checkout here: ");
            generateBill(fruits, fruitIdx, price, quantity);
        }

    }
}
```

Question 6

Section	Description
1. Problem	The task involves creating a Java program that can perform the following operations on a square matrix (2D array) of integers: **Input a Matrix:** The program should prompt the user to enter an integer n representing the dimensions of the n x n matrix. It should then take n x n integers as input to form the matrix. **Display the Original Matrix:** After taking the input, the program should display the matrix in a formatted way. **Rotate the Matrix:** The program should rotate the matrix 90 degrees clockwise and display the rotated matrix.
2. Solution	There are three methods to handle matrix input, display, and rotation: **parseMatrix**: • Prompts the user for an integer n which is the matrix dimension. • Collects n x n integers row by row and stores them in a 2D array (matrix) • Returns the matrix with values filled in for further processing. **displayMatrix**: • Takes a 2D array and displays it in a formatted way. • Ensures each element is printed neatly with proper spacing. **rotateMatrix**: • Rotates the given matrix 90 degrees clockwise. • The rotation strategy is to move each element from position [i][j] to [n-1-j][i]. • Displays the rotated matrix using the same formatting as displayMatrix. In the main method, the program executes these steps in sequence: • Parse the matrix using parseMatrix. • Display the original matrix using displayMatrix. • Rotate and display the matrix using rotateMatrix.

3. Sample Input & Output

```
Enter the dimension of the square matrix: 3
Enter row number 1: 1 2 3
Enter row number 2: 4 5 6
Enter row number 3: 7 8 9
The original matrix:
 1  2  3
 4  5  6
 7  8  9

The rotated matrix:
 7  4  1
 8  5  2
 9  6  3
```

```
Enter the dimension of the square matrix: 4
Enter row number 1: 1 2 3 4
Enter row number 2: 5 6 7 8
Enter row number 3: 9 10 11 12
Enter row number 4: 13 14 15 16
The original matrix:
 1  2  3  4
 5  6  7  8
 9 10 11 12
13 14 15 16

The rotated matrix:
13  9  5  1
14 10  6  2
15 11  7  3
16 12  8  4
```

4. Source Code

Link: Q6Vosshin.java

```
package com.mycompany.viva2;

import java.util.Scanner;

public class Q6Vosshin {

    public static  int[][] parseMatrix(){
        Scanner sc = new Scanner (System.in);
        System.out.print("Enter the dimension of the square matrix: ");
        int n = sc.nextInt();
        int [][] matrix = new int[n][n];
        for (int i=0;i<n;i++){
            System.out.printf("Enter row number %d:", i+1);
            for (int j = 0; j < n; j++) {
                matrix [i][j] = sc.nextInt();
            }
        }

    return matrix;
    }//parseMatrix method




    public static void displayMatrix(int[][] matrix){
```

```
        System.out.println("The original matrix: ");
        for (int i = 0; i < matrix.length ; i++) {//signifies row
            for (int j = 0; j < matrix[0].length; j++) { //signifies column
                System.out.print (((matrix[i][j]<10) ? " " + matrix[i][j] : matrix[i][j]) + " ");
            }
            System.out.println();
        }
        System.out.println();
    }//displayMatrix method

    public static void rotateMatrix(int[][] matrix){
        int n = matrix.length;

        // transpose
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }

        // reverse
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n / 2; j++) {
                int temp = matrix[i][j];
                matrix[i][j] = matrix[i][n - 1 - j];
                matrix[i][n - 1 - j] = temp;
            }
        }

        System.out.println("The rotated matrix: ");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(((matrix[i][j] < 10) ? " " + matrix[i][j] : matrix[i][j]) + "
");
            }
            System.out.println();
        }
    }
    public static void main (String [] args){
    int matrix[][] = parseMatrix();
    displayMatrix(matrix);
    rotateMatrix(matrix);
    }
}//class
```